

BuildHub AI

An agentic AI that keeps web applications healthy and easy to use - it repairs runtime failures and improves confusing UI, and never changes anything risky without a human decision.

One line: BuildHub AI watches a live web app, detects **bugs** (server errors) and **usability problems** (users who cannot find a button), uses a team of AI agents to design a fix, **proves the fix** (live validation for bugs, a sandbox with simulated users for UI), and asks a human for approval in **Jira** before any risky change reaches production.

Item	Detail
Category	AI for software reliability, DevOps (AIOps) and digital user experience
Primary users	Small and mid-sized software teams, startups, agencies and IT teams of MSMEs / institutions that run web apps without a 24x7 SRE team
Status	Working prototype: two AI pipelines, sandbox test environment with live view, Jira + Command Center + email approvals, automated test suites
Source code	https://github.com/Gokul-pixel-1/teachnova26 (public)
Stack	Next.js 16, TypeScript, PostgreSQL + Prisma, Groq-hosted LLM (qwen3.8-27b), Puppeteer + Chrome, Jira Cloud REST API, Gmail API, Telegram

How to read this document. Sections follow the jury evaluation criteria (1-11) and the Grand Finale questions. Everything described as "built" was run and tested on the prototype. Items marked **[Team to complete]** are field-validation evidence the team must collect and insert before the finale; we deliberately did not invent survey or interview numbers.

Contents

- Executive summary
- 1. Problem identification and national relevance
- 2. User understanding and problem validation
- 3. Solution concept and innovation
- 4. Why AI - relevance and depth of the AI application
- 5. Technical feasibility and architecture
- 6. Prototype and live demonstration
- 7. Testing, accuracy and performance evidence
- 8. Responsible AI, privacy, security and inclusiveness
- 9. Impact, scalability and sustainability
- 10. Business model, go-to-market and competitive advantage
- 11. Team capability and 12-month roadmap
- 12. Jury questions - prepared answers
- 13. Known limitations (honest status)
- Appendix - how to run and verify

Executive summary

Every web application fails in two ways. It **breaks** (a server error stops users from logging in, posting or paying), and it **confuses** (a button is in the wrong place, so users click empty space, give up, or call support). Large companies handle both with 24x7 on-call engineers, observability suites and UX research teams. Most small teams, startups, colleges, government-service vendors and MSMEs have none of these: a bug is found when a customer complains, and a confusing screen is never found at all.

BuildHub AI is an agentic AI layer that closes both loops end-to-end:

- **Pipeline 1 - Self-healing.** Server errors become incidents automatically. Four AI agents (Analyzer, Coder, Critic, Judge) find the root cause in the real source file and write a patch. A deterministic risk engine decides: LOW risk is applied, validated with live requests and rolled back automatically if validation fails; MEDIUM/HIGH risk (for example authentication code) stops and waits for a human.
- **Pipeline 2 - UX Suggestion Agent.** A lightweight tracker records anonymous interaction signals (clicks on empty space, rage clicks, slow finds) per UI component. When enough distinct users struggle, the AI learns *where* users expected the control (direction and click hotspot), proposes a code change, and **tests it first in a sandbox copy of the app** where simulated users replay the real clicks. Only a placement that passes (users find it where they looked, it is closer, nothing overlaps or duplicates) is offered for approval; otherwise the AI tries another placement (up to 3).
- **Human control in the tools teams already use.** Approval requests appear as **Jira cards** (moving a card to Done approves, To Do rejects), with the evidence, code diff and before/after screenshots attached. The Command Center dashboard and email remain available as fallback.

What is proven in the prototype	Evidence
Automatic detection -> diagnosis -> repair -> validation -> rollback	LOW-risk fault repaired automatically; HIGH-risk fault held for approval, approved in Jira, resolved (verify-repair-flow: 15/15 checks)
Behaviour-driven UX suggestions tested in a sandbox before approval	verify-ux-suggestions: 41/41 checks; real Groq runs where round 1-2 failed and round 3 passed
Jira approval for both pipelines	verify-jira-approval: 44/44 checks (approve, reject by comment, expiry, dashboard mirror, screenshots attached)
Both pipelines run concurrently without interference	Old and new test suites run in parallel on one server: all checks passed

Why it matters now: India is digitising services at population scale while most software is built and maintained by small teams. Reliability and usability are no longer "nice to have" - a failed login or an unusable form directly blocks a citizen, a student or a customer. Generative AI can now read real code and propose correct patches, which makes an affordable "AI site-reliability + UX engineer" possible for the first time.

1. Problem identification and national relevance

1.1 What problem are we solving?

Two connected problems in running web applications:

Problem	What happens today	Consequence
A. Runtime failures (bugs)	An error is logged; nobody reads the logs until a user complains. An engineer must reproduce it, find the file, write a fix, test and deploy - often hours or days later.	Downtime, lost transactions, loss of trust, stressed on-call engineers, high Mean Time To Resolution (MTTR).
B. Invisible usability friction	Users click where they expect a control, find nothing, click repeatedly ("rage clicks"), or take a long time. Analytics tools can show heatmaps, but a human must interpret them, redesign, and ship.	Abandoned sign-ups and forms, support calls, exclusion of less-experienced users. Most small teams never run UX research at all.
C. Unsafe automation	Teams that try automation either (a) only get alerts, or (b) fear letting a bot change production code without control.	AI remains a "suggestion box"; the loop from detection to verified fix is still manual.

1.2 For whom?

- **Primary:** small and mid-sized software teams (2-50 engineers) - startups, SaaS products, digital agencies and IT service vendors - who run production web apps without a dedicated 24x7 SRE or UX research team.
- **Secondary:** IT teams of MSMEs, colleges/ed-tech platforms and vendors building e-governance or citizen-service portals, where downtime or a confusing form directly blocks the public.
- **Beneficiaries:** the end users of those applications (customers, students, citizens), especially first-time and less digitally-confident users who suffer most from confusing UI.

1.3 Why does it matter now - importance to India

- **Population-scale digital services.** Banking, payments, education, health and government services in India are increasingly delivered through web and mobile apps. When they fail or confuse, the impact is on millions of people, many of them new to digital services.
- **Talent is concentrated in large companies.** Experienced SRE, DevOps and UX specialists are scarce and expensive; smaller firms and public-sector vendors rarely afford them. AI can extend their expertise to every team.
- **MSME and startup digitisation.** Many small businesses now run their own web storefronts, portals and apps with lean teams; they need reliability and usability without enterprise tooling costs.
- **Generative AI has crossed a threshold.** Language models can now read a real source file, reason about a stack trace and propose a precise edit. Combined with deterministic safety checks, this makes autonomous-but-controlled repair practical.
- **Inclusion.** Usability friction hurts first-time, elderly and non-English-first users most. Detecting it from behaviour (not from surveys) surfaces problems that these users never report.

1.4 Urgency and scale

Every minute of a broken login or payment flow is lost revenue and trust; every confusing button repeats its cost for every visitor. The problem scales with the number of apps and users, not with the number of engineers - which is exactly why a human-only approach does not keep up.

1.5 Evidence that the problem exists

Type of evidence	What we have	Status
Industry observation	The existence and growth of whole product categories - error monitoring, on-call/incident management, session-replay and heatmap tools - shows teams pay to detect these problems. Yet these tools stop at detection; fixing remains manual.	Observed
Behavioural signals are real and measurable	Rage clicks and dead clicks are standard metrics in session-analytics tools. Our prototype captures them per component and turns them into a friction score.	Built and demonstrated
Prototype evidence	In our own app, a demo fault left permanently active in a commit silently broke post creation (every post returned HTTP 500) until an automated test run exposed it - a real example of a bug nobody noticed.	Observed in this project
User interviews	[Team to complete] Interview 8-15 developers / founders / IT leads (questions in Section 2.4).	Planned
Survey	[Team to complete] Short online survey of developers in college and startup communities (questions in Section 2.4).	Planned

2. User understanding and problem validation

2.1 User personas

Persona	Context	Pain today	What they need from BuildHub AI
Priya - startup CTO	5-person team, one production SaaS, no on-call rotation	Wakes up to customer complaints; fixes bugs at night; no time for UX research	Safe automatic fixes for small bugs; one Jira card to approve risky ones; UI issues surfaced automatically
Ravi - agency developer	Maintains 20+ client websites	Cannot watch every site; clients report problems late	One dashboard across sites; evidence-rich tickets; proof before any change
Meena - IT lead, college / institution portal	Admissions, results and fee portals with seasonal peaks	Outages at peak time; students confused by forms	Fast recovery during peaks; clear audit trail; human approval for anything touching login or data
Arjun - product manager	Owens sign-up / checkout conversion	Heatmaps show rage clicks but not what to change	A tested design suggestion with before/after screenshots and the predicted benefit
End user (citizen / student / customer)	Uses the app, often on mobile, sometimes a first-time user	Cannot find the button; errors with no explanation	An app that works and gets easier over time - without having to complain

2.2 User needs we designed for

- **Trust before automation** - users will not let a bot edit production code blindly. Hence: deterministic risk tiers, validation, automatic rollback, and human approval for risky changes.
- **Evidence, not guesses** - every suggestion shows the real error / real user behaviour, the exact code before and after, and (for UI) sandbox test results and screenshots.
- **Fit into existing workflow** - approvals live in Jira (where teams already plan work), not in yet another tool. Email and dashboard remain for teams without Jira.
- **Low effort to adopt** - tracking a UI element needs one attribute (data-ux-id); bug detection works from the existing server logs.

2.3 Existing solutions and their gaps

Existing approach (examples)	What it does well	Gap BuildHub AI fills
Error monitoring / APM (e.g. Sentry, Datadog, New Relic)	Captures errors, traces and performance; some now add AI explanations or suggested fixes	Stops at detection/suggestion; the verified apply-validate-rollback loop and risk-gated approval are left to engineers
Incident management (e.g. PagerDuty, Opsgenie)	Alerts and on-call routing	Wakes a human; does not diagnose or fix code
AI coding assistants (e.g. GitHub Copilot and similar)	Help a developer write or fix code on request	Need a developer to notice the problem, prompt, test and deploy; not connected to live production signals
Session analytics / heatmaps (e.g. Microsoft Clarity, Hotjar)	Show rage clicks, dead clicks, heatmaps, recordings	A human must interpret and redesign; no code change, no test of the new design
A/B testing platforms (e.g. Optimizely, VWO)	Measure design variants with real traffic	Someone must design variants; testing on real users is slow and risky; small sites lack traffic

Existing approach (examples)	What it does well	Gap BuildHub AI fills
Manual UX research	Deep qualitative insight	Expensive and slow; rarely done by small teams

Our gap statement: no widely-used tool closes the loop *detect -> diagnose -> write the fix -> prove it works -> get a human decision -> apply -> verify / roll back* for **both** runtime bugs and UI usability, in one place, with the approval inside the team's existing Jira workflow.

2.4 Validation plan [Team to complete before the finale]

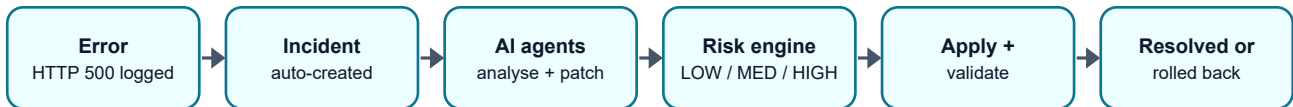
To move beyond assumptions, the team should collect and insert real evidence here:

Method	Target	Key questions / measures
User interviews	8-15 developers, startup founders, agency owners, college IT leads	1) How do you find out a production bug happened? 2) Time from bug to fix last time? 3) Would you allow an AI to auto-fix low-risk bugs? What would you require? 4) Have you ever found a confusing UI element? How? 5) Where do you want approvals - Jira, email, Slack, dashboard?
Survey (Google Form)	50-100 respondents from developer communities, college tech clubs	Team size; on-call arrangement; monitoring tools used; average MTTR; trust level for AI fixes (1-5); willingness to pay (range)
Pilot	1-3 real web apps (college fest site, a startup, an NGO portal)	Incidents detected, % auto-fixed, time-to-resolution vs before, UX suggestions accepted, rage-click reduction after change
Industry interaction	Mentors, DevOps engineers, product managers	Feedback on risk policy and on what evidence they need to approve a change

Suggested insight format for the slide: "*X of Y developers interviewed said...*" - use only numbers the team actually collects.

3. Solution concept and innovation

3.1 Solution journey - Pipeline 1: self-healing for runtime failures



- **Detection:** the observability logger captures every server error with stack trace and source location. A log monitor groups identical failures into one incident (no duplicate tickets).
- **Multi-agent reasoning:** Analyzer (root cause from real evidence) -> Coder (patch against the real file) -> Critic (independent review, security and regression concerns) -> Judge (approve / reject with confidence). Up to 3 conversation rounds.
- **Deterministic risk gate:** the AI does not decide its own authority. A rule-based engine classifies risk by file, route and impact (e.g. authentication = HIGH).
- **Validation and rollback:** before applying, the original bytes and SHA-256 are checkpointed; live HTTP probes replay the failing request; if any probe fails, the original is restored and verified.
- **Human approval:** MEDIUM/HIGH patches pause at WAITING_APPROVAL and create a Jira card with the full evidence. Done = apply; To Do = reject.
- **Learning memory:** every outcome is stored as an experience (state, action, reward) so repeated failures and human decisions inform later runs.

3.2 Solution journey - Pipeline 2: UX Suggestion Agent



- **Behaviour capture:** any element marked data-ux-id is tracked. The tracker records clicks on empty space near it (with exact offset and direction), rapid repeated clicks, and time-to-find. Only anonymous session ids - no personal data.
- **Friction analysis:** score = 2 x dead clicks + 3 x rage clicks + 2 x slow finds; a suggestion is raised only when the score reaches 12 **and** at least 3 distinct users struggled (one confused person is not a pattern). It also computes the dominant direction (left/right/up/down) and the median click hotspot.
- **AI drafting:** the LLM receives the real source file, the element, the direction and hotspot, and must return an exact, verbatim-anchored code edit that *moves* the element (restyling alone is rejected by the sandbox).
- **Sandbox testing:** the change is written only into a separate copy of the app (port 3100). A Chrome browser replays every real user's expected click on the new layout and measures: found-where-looked rate ($\geq 60\%$), distance improvement ($\geq 50\%$), visibility, clickability, size ≥ 24 px (WCAG), no overlap, no duplicate control, no misclick on a neighbouring button, no horizontal overflow, and page still loads.
- **Iterative search:** if a placement fails, the measured reasons ("did not move", "users would press Sign up by mistake") are fed back, and the AI must propose a genuinely different placement - up to 3 rounds. If none pass, nothing is sent and nothing changes.
- **Live view:** the sandbox can run in a visible window where a cursor replays users, marks each spot (found / missed / misclick), and a status bar narrates each round - ideal for demos and for trust.

3.3 What is original

Innovation	Why it is new / better
One agentic loop for bugs AND usability	Existing tools treat reliability and UX as separate products. BuildHub uses the same detect -> reason -> prove -> approve -> verify pattern for both.
"Prove before you propose" for UI	UX changes are tested with simulated users that replay real behaviour <i>before</i> a human sees them. Humans review only placements that already passed.
Behaviour-derived design intent	Direction + click hotspot tell the AI <i>where</i> users expect a control, turning heatmap data into an actionable design constraint.
AI proposes, rules decide	LLM creativity is bounded by a deterministic risk engine, verbatim-anchored patches, SHA-256 integrity checks and automatic rollback.
Approval inside Jira	Decisions happen where teams already work; results are written back as comments; decisions made elsewhere are mirrored to the card.

3.4 Value proposition

For small software teams who cannot afford 24x7 SREs or UX researchers, **BuildHub AI** is an AI reliability-and-usability engineer that **finds problems from real signals, writes and proves the fix, and asks for approval in Jira** - unlike monitoring and analytics tools, which stop at telling you something is wrong.

4. Why AI - relevance and depth of the AI application

4.1 Why the solution cannot work without AI

- **Fixes must be written for code that nobody pre-programmed.** A rule engine can detect "HTTP 500 on /api/posts"; only a language model can read an arbitrary stack trace and source file, explain the root cause, and write a correct, minimal edit in that codebase's style.
- **UI redesign is open-ended.** "Users look for Log in on the left" has many possible code answers (reorder, change alignment, move out of a group, new row). Generating valid TSX edits for any component requires generative AI; templates would only work for elements we anticipated.
- **Learning from failure.** The AI takes measured sandbox feedback in natural language ("the element did not move; users would press Sign up") and changes strategy - a reasoning loop, not a lookup.
- **Review and judgement.** Critic and Judge agents evaluate security and regression concerns of a patch in context - a form of reasoning that static linters cannot do.

4.2 Intelligence created (beyond an API call)

Layer	Type of AI / method	Intelligence produced
Behaviour analytics	Signal processing + statistical aggregation (friction score, dominant direction, median hotspot, distinct-user threshold)	Which component is hard to use, for how many users, and where they expected it
Root-cause analysis	LLM reasoning over logs, stack traces and real source (Analyzer agent)	Explained cause and exact location (file, line, function)
Code generation	Generative AI (Coder / UX reviewer), verbatim-anchored patch format	A minimal, applicable code edit
Multi-agent critique	Critic + Judge agents, up to 3 rounds	Security / regression review and a confidence-scored decision
Simulation-based evaluation	Headless-browser simulation replaying recorded user expectations; Fitts's-law time model; WCAG checks	Predicted usability benefit of a design before release
Iterative optimisation	Feedback loop: measured failure reasons -> new AI proposal, no repeats	A search over design placements that converges on one that passes
Learning memory	Reinforcement-style experience records (state, action, reward) + human decisions	Repair memory that informs future decisions

4.3 Decisions, predictions and automation enabled by AI

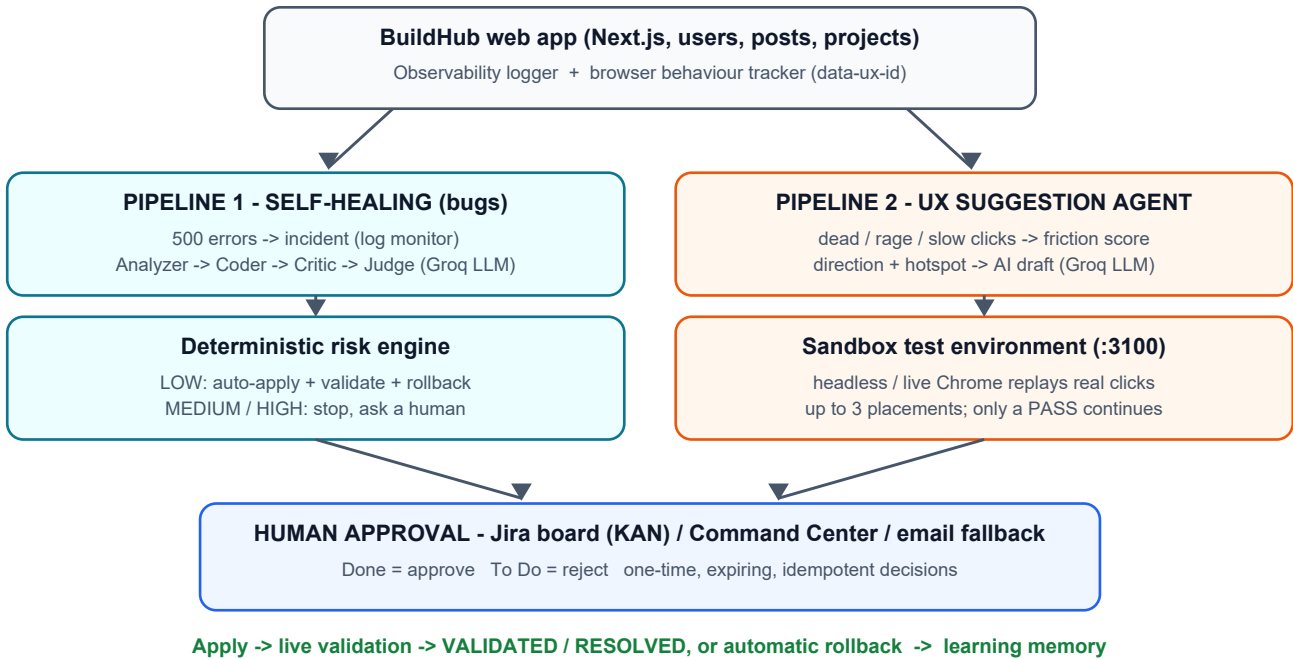
- **Decides** what is broken and where; which patch to propose; whether a patch is acceptable (Judge); which UI placement to try next.
- **Predicts** whether a new layout will be easier: share of users who will find the element where they look, distance reduction, predicted time-to-find.
- **Automates** the full LOW-risk repair loop and all preparatory work for risky ones, so a human only makes the final yes/no decision with evidence in hand.

4.4 Model selection

The prototype uses a Groq-hosted open-weight model (qwen3.8-27b) for fast, low-cost inference; the provider layer is pluggable (a local Ollama model and a deterministic offline TEST provider are also supported). An open-weight model allows on-premise deployment for customers who cannot send code outside their network (government, banking, education). Correctness does not rely on the model alone: every output is structurally verified, anchored to the real file and tested before it can take effect.

5. Technical feasibility and architecture

5.1 System architecture



5.2 Components

Component	Technology	Role
Web application	Next.js 16 (App Router), React, TypeScript, Tailwind	The monitored product (developer collaboration platform) and the AI Command Center
Data store	PostgreSQL + Prisma 7 ORM	Logs, incidents, repair attempts, patches, approvals, UX events, UX suggestions, Jira links, learning memory
AI provider layer	Groq API (qwen3.8-27b); Ollama; deterministic TEST provider	Analyzer / Coder / Critic / Judge and UX reviewer prompts
Observability	Structured logger, log monitor, auto-scan	Turns 500 errors into incidents; groups duplicates
Behaviour tracker	Browser script, data-ux-id attribute, batched events API	Anonymous dead / rage / slow-find events with offsets
Sandbox	Separate copy of the app on port 3100, Next.js dev server, no credentials	Safe place to render candidate UI changes
Simulator	Puppeteer-core + Chrome/Edge (headless or live window)	Replays user expectations, measures layout, screenshots
Approvals	Jira Cloud REST v3 (poller), Command Center, Gmail API (OAuth), Telegram	Human-in-the-loop decisions with one-time, expiring approvals
Safety	Deterministic risk engine, SHA-256 checkpoints, rollback, operator auth guard	Guarantees no unapproved risky change and recoverability

5.3 Data required

- **Bugs:** server error logs with stack traces (already produced by the app) and read access to the source repository. No user personal data is needed.

- **UX:** anonymous interaction events per tracked component - event type, click offset from the element, viewport size, time-to-find, random session id. No names, keystrokes, form contents or recordings.
- **No training dataset is required to start:** the system uses a pre-trained LLM plus the app's own live signals, so it works from day one on a new application; learning memory improves with use.

5.4 Deployment feasibility and scalability

- **Today:** runs on a single laptop (Windows) with embedded PostgreSQL - proven during the hackathon.
- **Production path:** (1) SDK / middleware for the customer app (logger + tracker), (2) BuildHub cloud or on-prem service (analysis, agents, approvals), (3) repository integration that opens a pull request instead of writing files directly, (4) sandbox as an ephemeral preview environment per suggestion (containers / preview deployments).
- **Scaling:** stateless API + queue workers for agent runs and simulations; one sandbox per job in containers; Postgres with partitioned event tables; LLM calls are the main cost and are made only when a threshold is crossed (not per event).

6. Prototype and live demonstration

Demo storyline: **User problem -> Solution journey -> AI functionality -> Output / impact**. Total about 7-8 minutes; keep explanation short and let the system act.

Step	What the jury sees	Time
1. Setup	Four windows: BuildHub site (localhost:3000), Command Center (/ai), Jira board KAN, live sandbox window (3100).	0:30
2. Small bug (LOW)	Switch on demo fault LOW-01 -> Publish a post fails (HTTP 500) -> incident appears -> AI agents analyse -> patch applied and validated -> RESOLVED automatically -> posting works. No human needed.	1:30
3. Big bug (HIGH)	Switch on HIGH-01 (password checker crashes) -> wrong-password login gives a server error -> AI prepares a fix but stops (auth code = HIGH risk) -> Jira card appears in In Review with root cause and before/after code -> drag to Done -> BuildHub comments "patch applied and validation passed" -> login answers correctly again.	2:00
4. UX friction	Three users (three fresh tabs) click empty space to the right of "Explore projects", rage-click, then find it late -> dashboard shows friction score crossing the threshold.	1:00
5. Sandbox, live	Live window: cursor replays the users on the original page (red marks), AI round 1 idea applied only in the sandbox, users replayed again, result shown; if it fails, round 2/3 try a different placement until one passes (green marks).	1:30
6. Approval + impact	Jira card with sandbox results and before/after screenshots -> drag to Done -> the real site now shows the button where users looked; card gets the result comment.	1:00
7. Safety close	Show a rejected card (To Do) - nothing changed; show the incident timeline and audit trail.	0:30

6.1 Screens and outputs to show

- Command Center: system health, live AI feed, incident story (Analyzer -> Coder -> Critic -> Judge), learning memory.
- UX Suggestions page: friction per component, direction and hotspot, sandbox rounds with before/after screenshots, re-test button.
- Jira card: evidence, code diff, sandbox rounds, attached screenshots, BuildHub result comment.
- Fallbacks if the network fails: the TEST provider runs the whole flow offline and deterministically; the mock Jira server reproduces Jira locally.

Demo tips: start the app and sandbox 10 minutes before; run one warm-up UX test (the first sandbox compile is slow); keep the free-tier AI rate limit in mind (the system waits and retries automatically); have screenshots of a successful run as backup.

7. Testing, accuracy and performance evidence

7.1 Automated test suites (all run on the prototype)

Suite	What it proves	Result
verify-ux-suggestions	Below-threshold behaviour raises nothing; struggle raises a suggestion; sandbox tests before approval; screenshots served; approve -> VALIDATED; misclick layouts -> NO_EASY_PLACEMENT with no approval sent; re-test never repeats a placement; reject / expiry / bogus token leave files untouched; file restored byte-for-byte	41/41 (Jira channel), 42/42 (email channel)
verify-repair-flow	Posting works; LOW-01: failure -> incident -> automatic repair -> RESOLVED; HIGH-01: failure -> incident -> waits for approval -> approved -> RESOLVED; posting and login work afterwards	15/15 (both channels)
verify-jira-approval	Jira status (no token exposed); repair approve via Done; "In Progress" decides nothing; reject via comment by background poller; dashboard decision mirrored to Jira; expired approval never executes; UX card with sandbox results and before/after screenshots; UX approve and reject	44/44
Concurrency	Old (bug-repair) and new (UX) suites executed in parallel against the same server	All passed
Type safety / lint	TypeScript compiler and ESLint on all new code	Clean

7.2 Accuracy and quality measures

Measure	How it is enforced / measured
Patch correctness	Verbatim anchoring to the real file, structural verification, live validation probes replaying the failing request; failure -> automatic rollback with SHA-256 verification. A wrong patch cannot stay applied.
UX suggestion quality	Only placements with $\geq 60\%$ of simulated users finding the element where they looked AND $\geq 50\%$ closer AND all layout checks passing are offered. Thresholds are configurable.
False alarms (UX)	Requires friction score ≥ 12 from ≥ 3 distinct users; single confused users are ignored (tested).
Observed AI behaviour	In real Groq runs, round-1 proposals sometimes only restyled or reordered the element; the sandbox correctly rejected them and a later round passed (e.g. round 3: 18/18 checks, 78-95% closer). This shows why testing before approval is necessary.

7.3 Error handling and edge cases

- AI provider rate limit (HTTP 429): waits and retries automatically.
- AI proposal that breaks the page: sandbox page fails to load -> round fails, trial continues; real site untouched.
- Proposal that duplicates a control or causes misclicks: detected and rejected with a reason fed back to the AI.
- Approval clicked twice, after expiry, or with a bogus token: refused; nothing executed twice.
- Jira unreachable: falls back to email so a decision request is never lost.
- Stale sandbox trials expire after 20 minutes; one sandbox trial runs at a time.

7.4 Comparison with existing approaches

Step	Manual process	BuildHub AI
Notice a bug	When a user complains	Seconds after the first 500 error
Find the cause	Engineer reproduces and debugs	Analyzer agent with real evidence and location
Write + review the fix	Engineer + reviewer	Coder + Critic + Judge agents
Safe release	Manual testing, manual rollback	Risk gate, live validation, automatic rollback
Notice a confusing UI	Often never	Automatically, from behaviour of 3+ users
Test a new design	A/B test on real users (days, needs traffic)	Sandbox simulation in minutes, before any user sees it

Field metrics to collect in pilots [Team to complete]: MTTR before/after, % incidents auto-resolved, % suggestions approved, rage-click reduction after an approved UX change.

8. Responsible AI, privacy, security and inclusiveness

Principle	How BuildHub AI implements it
Human oversight	Risky code changes and every UI change require an explicit human decision (Jira / dashboard / email). The AI never grants itself authority; a deterministic engine decides risk.
Explainability	Every decision request shows the evidence (error or behaviour), the AI's root cause, the exact before/after code, the reviewer's concerns and, for UI, per-round sandbox results and screenshots. Structured summaries, no hidden chain-of-thought needed.
Reversibility	SHA-256 checkpoints and automatic rollback; audit trail of every event, approval and outcome.
Privacy by design	UX tracking stores only anonymous random session ids, event types, click offsets and viewport sizes - no personal data, no keystrokes, no recordings. Bug repair uses logs and code, not user data.
Secret handling	API keys and tokens live only in local environment files (never committed, never logged, never shown by status APIs). The sandbox runs with no credentials, so it can never email, alert or call AI. OAuth codes are kept out of request logs. A secret scan was run before publishing the repository.
Security	Operator-only APIs, one-time hashed approval tokens with expiry, idempotent decisions, TEST mode can only talk to a local mock Jira, security log analyzer for attack patterns.
Fairness and bias	Friction thresholds require several distinct users, so one user cannot drive a change. Simulation replays every recorded user and uses the viewport sizes users actually had (including mobile), so designs are not optimised only for desktop users.
Accessibility	Every UI candidate must keep targets ≥ 24 px (WCAG 2.2 minimum target size), stay visible, clickable and not overflow on small screens. Future: contrast and keyboard-navigation checks.
Inclusiveness (languages / regions)	Behaviour signals are language-independent - they work for Hindi, Tamil or any-language interfaces without translation. Roadmap: multilingual summaries on Jira cards and dashboards.

9. Impact, scalability and sustainability

9.1 Measurable impact

Impact type	Metric	How BuildHub AI moves it
Economic (for teams)	Engineer hours per incident; MTTR; downtime minutes	Automatic LOW-risk fixes; ready-to-approve patches for the rest
Economic (for their business)	Conversion, sign-up and task completion rates	Removing friction where users actually struggle
Social	Successful completion of digital services by first-time users	Detects confusion that such users never report
Industry	Reliability and UX capability for small firms	Enterprise-grade practices without enterprise headcount
Environmental (secondary)	Wasted compute from retry storms and failing requests	Faster resolution reduces repeated failing traffic

9.2 Beneficiaries

- Developers and founders of startups and SMEs; digital agencies; IT teams of colleges, hospitals and NGOs; vendors of e-governance portals.
- End users: customers, students and citizens who get working, easier apps.

9.3 Scalability across sectors and geographies

- **Sector-agnostic core:** any web app with logs and a DOM can use it - e-commerce, ed-tech, fintech, health, government services.
- **Geography-agnostic signals:** behaviour and error signals do not depend on language or region.
- **Technology expansion:** Next.js/React first, then other web frameworks, then mobile apps (tap events) and backend services in other languages.

9.4 Long-term sustainability

- Recurring SaaS revenue funds the service; on-prem licences for regulated customers.
- Learning memory and a growing library of verified fixes and UX patterns increase value with every customer (data network effect, anonymised).
- Pluggable models avoid lock-in to one AI vendor and allow cost optimisation as model prices fall.

10. Business model, go-to-market and competitive advantage

10.1 Market - who are the users and customers?

Segment	User	Payer	Priority
Early-stage startups and SaaS teams (2-50 engineers)	Developers, CTO	Company (CTO / founder)	1 - beachhead
Digital agencies and IT service firms	Developers maintaining client sites	Agency (resold to clients)	2
Colleges, ed-tech and institution portals	IT team	Institution	3
Mid-size enterprises and e-governance vendors	SRE / platform teams	Enterprise / vendor (on-prem)	4

First target: Indian startups and small SaaS/product teams building on React/Next.js - they feel the pain most (no on-call team), decide quickly, already use Jira and GitHub, and can install an SDK in a day.

10.2 Why will they adopt it?

- Immediate value on day one - no training data needed; the first real error or struggle produces a ready fix.
- Low risk - nothing risky changes without their approval; everything is reversible.
- Fits current workflow - Jira cards, GitHub pull requests, email.
- Visible proof - sandbox screenshots and validation results make approval easy.

10.3 Adoption - how users discover and start

- **Open-source core + free tier** for one project: logger, tracker and dashboard free; AI fixes limited per month.
- **Developer channels:** GitHub repository and README, npm package, technical blog posts and demo videos ("watch the AI fix a bug live"), hackathons, college tech clubs, startup communities and incubators.
- **Integrations marketplace:** Jira / Atlassian Marketplace app, GitHub App, Vercel / Netlify integration.
- **Onboarding in under 30 minutes:** install package, add data-ux-id to key buttons, connect Jira.

10.4 Business model - who pays and how (proposed, to be validated)

Plan	For	Includes	Pricing model
Free	Students, hobby projects	1 app, monitoring + UX analytics, limited AI fixes / suggestions per month	Free
Team	Startups, small teams	Several apps, unlimited detection, monthly AI-fix and sandbox-test quota, Jira/GitHub	Per app per month (tiered)
Agency	Agencies, IT firms	Many client apps, white-label reports, multi-tenant dashboard	Volume-based per app
Enterprise / Gov	Regulated organisations	On-prem / private-cloud deployment, local open-weight model, SSO, audit, SLA	Annual licence + support

Exact prices are hypotheses to test in pilots (willingness-to-pay question in the survey). Main variable cost = LLM inference and sandbox compute per suggestion, which is incurred only when a threshold is crossed, keeping margins healthy.

10.5 Reaching the first 1,000 users

Phase	Action	Target
Month 0-2	Pilot with our college portals, test websites and 3-5 startups from incubator networks; publish case studies	10-20 teams
Month 2-4	Open-source launch (GitHub + npm), demo video, developer community posts, college tech-club workshops	200 users
Month 4-6	Atlassian Marketplace and GitHub App listing; hackathon sponsorship; referral credits (extra AI fixes)	500 users
Month 6-9	Agency partner programme; content on "AI that fixes your site"; integrations with hosting platforms	1,000+ users

10.6 Competitive advantage and IP potential

- **Closed loop with proof:** detection -> AI fix -> validation / sandbox simulation -> approval -> verification, for both bugs and UX.
- **Simulation-based UX evaluation** using replayed real behaviour - potential for patent / IP around "behaviour-replay validation of AI-generated UI changes".
- **Safety architecture** (deterministic risk gate, verbatim anchoring, rollback) that makes autonomous changes acceptable to cautious teams.
- **Data advantage over time:** anonymised library of verified fixes and UX patterns.
- **Cost and sovereignty:** open-weight models allow on-prem deployment for Indian regulated sectors.

11. Team capability and 12-month roadmap

11.1 Team capability

[Team to complete: names, roles, skills.] The prototype demonstrates capability in full-stack web development (Next.js, TypeScript, PostgreSQL), LLM integration and prompt engineering, multi-agent design, browser automation and simulation, DevOps safety engineering (validation, rollback), third-party integrations (Jira, Gmail OAuth, Telegram) and automated testing.

11.2 Roadmap

Quarter	Product	Business
Q1 (months 1-3)	Package logger + tracker as npm SDK; GitHub pull-request mode instead of direct file writes; containerised per-job sandbox; Slack approval channel; multilingual card summaries	5-10 pilots; collect interviews, survey and pilot metrics; refine pricing
Q2 (months 4-6)	Hosted multi-tenant cloud; Atlassian Marketplace + GitHub App; more UX checks (contrast, keyboard navigation, mobile tap targets); broader fault classes	Open-source launch; first paying Team customers; 500 users
Q3 (months 7-9)	Support for other frameworks (Vue, Angular, plain HTML); mobile app events; learning-based risk suggestions from approval history	Agency partner programme; 1,000+ users; incubator / grant applications
Q4 (months 10-12)	On-prem enterprise edition with local open-weight model, SSO, audit export; SOC-style security review	First enterprise / institutional contracts; seed funding round

12. Jury questions - prepared answers

Q: What makes your solution different from existing solutions?

Existing tools stop at detection (monitoring, heatmaps) or need a developer to drive them (AI coding assistants). BuildHub AI closes the whole loop - it detects from live signals, writes the fix, proves it (live validation for bugs, simulated users in a sandbox for UI), gets a human decision in Jira, applies it and verifies or rolls back. And it does this for both reliability and usability.

Q: What happens if the AI model fails?

Nothing unsafe. A malformed or non-matching patch is rejected before it is applied. A LOW-risk patch that fails validation is rolled back automatically (SHA-256 verified). Risky patches and all UI changes need a human. A UI idea that fails the sandbox is never sent; the AI tries up to 3 different ideas, then stops with "no easy placement". If the AI service is down or rate-limited, the system waits/retries and the app keeps running normally - the AI is an add-on, not a dependency of the app.

Q: What data is required?

Server error logs and read access to the source code for bug repair; anonymous interaction events for tagged UI elements (event type, click offset, viewport size, time-to-find, random session id). No personal data, no training dataset needed to start.

Q: How accurate is your solution?

Accuracy is enforced rather than assumed: a change is only applied if it passes validation (bugs) or simulated-user thresholds (UI: $\geq 60\%$ find it where they looked, $\geq 50\%$ closer, all layout checks). Our automated suites pass 41/41, 15/15 and 44/44 checks. Real-world accuracy (share of AI proposals that pass and are approved) will be measured in pilots; in our real-model runs early proposals were often rejected by the sandbox and a later round passed - which is exactly what the safety loop is for.

Q: What is your implementation cost?

For a customer: install an SDK, tag key UI elements, connect Jira - under a day. For us: hosting (web + Postgres), sandbox compute per suggestion, and LLM inference only when a threshold is crossed. The prototype ran on a laptop with a free-tier hosted model; on-prem deployments can use an open-weight model on the customer's own hardware. [Team to add a costed estimate per 1,000 incidents / suggestions.]

Q: Who are your competitors?

Error monitoring / APM (Sentry, Datadog, New Relic), incident management (PagerDuty), AI coding assistants (GitHub Copilot and similar), session analytics (Microsoft Clarity, Hotjar) and A/B testing (Optimizely, VWO). Each covers one part; we combine detection, AI fixing, proof and approval in one loop, and integrate with rather than replace some of them.

Q: How will you acquire your first 1,000 users?

Pilots with college portals and incubator startups -> open-source core and free tier -> developer content and live-fix demo videos -> Atlassian Marketplace / GitHub App listings -> agency partners and referral credits (Section 10.5).

Q: What is your roadmap for the next 12 months?

SDK + GitHub pull-request mode + containerised sandbox (Q1), hosted cloud + marketplace apps (Q2), more frameworks and mobile (Q3), on-prem enterprise edition (Q4), alongside pilots -> 1,000 users -> first enterprise contracts (Section 11.2).

Q: Is it safe to let AI change production code?

Only within strict limits: the AI never decides its own risk level; only LOW-risk changes are automatic and they are validated and reversible; everything touching authentication, security or data, and every UI change, needs a human. In production mode we will open a pull request so the team's normal CI and review also apply.

Q: Why not just use heatmaps?

Heatmaps show that users struggle; they do not say what to change, write the code, or test the new design. BuildHub turns the same signals into a tested code change ready for approval.

Q: How do you avoid AI bias or one user driving changes?

A change needs several distinct struggling users; the simulation replays every user on the viewport sizes they used; accessibility checks keep controls large and visible; a human approves every UI change.

Q: Who pays?

The software team or company running the app (CTO / engineering manager budget), agencies on behalf of clients, and institutions / vendors for on-prem licences. End users never pay.

13. Known limitations (honest status)

Stating limitations clearly builds jury trust. Current prototype limits and the planned answer:

Limitation	Plan
Changes are written directly to the local source tree (development mode).	Production mode opens a GitHub pull request so normal CI and code review apply.
UX changes limited to layout / placement of tagged elements; when users are only "slow" (no click positions), the AI can only make the element more prominent, and simulation checks layout sanity rather than findability.	Add eye-line heuristics, contrast checks and optional short in-app micro-surveys.
Demo faults are injected on purpose for the demonstration; broader real-world fault classes need more testing.	Pilot on real apps; expand fault library; learning from approvals.
Free-tier AI rate limits slow down rounds.	Paid tier / local model; caching of repeated analyses.
Jira approvals are detected by polling (the app runs on localhost).	Jira webhooks in the hosted version.
Single sandbox runs one trial at a time.	Containerised sandboxes per job.
User-validation evidence (interviews, survey, pilot metrics) not yet collected.	Complete Section 2.4 before the finale.
The original bug-repair test script restarts servers with Linux-only commands.	A Windows-friendly equivalent (verify-repair-flow) is provided.

Appendix - how to run and verify

Task	Command / place
Start database + app (real AI)	powershell -ExecutionPolicy Bypass -File C:\Users\sugan\buildhub-tools\start-buildhub.ps1
Website / Command Center	http://localhost:3000 http://localhost:3000/ai (operator: arjun / demo password)
UX Suggestions dashboard	http://localhost:3000/ai/ux-suggestions
Sandbox (test environment)	http://localhost:3100 - live window opens automatically when UX_SIM_LIVE=true
Jira setup (one-time, token typed locally)	node scripts/jira-setup.mjs
Jira status check	http://localhost:3000/api/jira/status
Hermetic tests	start-buildhub.ps1 -TestMode then node scripts/verify-repair-flow.mjs and node scripts/verify-ux-suggestions.mjs
Jira tests (mock Jira)	node scripts/mock-jira.mjs ; start-buildhub.ps1 -TestMode -JiraMock ; node scripts/verify-jira-approval.mjs
Simulate UX struggle	node scripts/simulate-ux-struggle.mjs --component landing-hero-explore --side right --distance 150
Source code	https://github.com/Gokul-pixel-1/teachnova26

Key files

- lib/server/repair/* - incident engine, agents, risk, validation, rollback
- lib/server/ux/* - behaviour analysis, AI drafting, sandbox, simulator, trials, live view, approvals
- lib/server/jira/* - Jira client and approval poller
- components/ux/behavior-tracker.tsx - browser behaviour tracker
- PLAN.md, JIRA_APPROVALS.md, GMAIL_ALERTING.md - design and setup notes